

Modified PIANO: A Physics-Informed DeepONet Framework for Satellite-Based Precipitation Nowcasting

1 Overview

This methodology extends the original PIANO framework by replacing its CNN-based temporal neural operator (T-NO) with a **physics-informed DeepONet (PI-DeepONet)**. The velocity-estimation stage (V-NO) is kept unchanged. The goal of the modification is to move from a fixed-grid, fixed-horizon image-to-image predictor to a **continuous, resolution-independent, mesh-free operator** that can be queried at arbitrary spatial locations and lead times, while remaining physically consistent with an advection process.

The framework still factorizes the nowcasting problem into two sub-tasks:

1. **Velocity estimation** — infer the motion field driving the evolution of the precipitation/cloud field from satellite imagery.
2. **Temporal propagation** — advance the current atmospheric state forward in time using that velocity field.

Only sub-task 2 changes. Figure 1 illustrates the full modified pipeline.

2 Problem Formulation

Let:

- $I(x, y, t_0) \in \mathbb{R}^{H \times W \times C}$ be the satellite image stack (e.g. IR/WV channels) at the current time t_0 , over a spatial grid $(x, y) \in \Omega \subset \mathbb{R}^2$.
- $\mathbf{v}(x, y) = (u(x, y), w(x, y))$ be the 2D velocity (motion) field estimated from the imagery.
- $s(x, y, t)$ be the scalar precipitation/brightness-temperature field we want to predict at future time $t = t_0 + \Delta t$.

The task is to learn an operator

$$\mathcal{G} : (I(\cdot, \cdot, t_0), \mathbf{v}(\cdot, \cdot)) \mapsto s(x, y, t) \quad (1)$$

that can be evaluated at **any** query coordinate (x, y, t) , not just the coordinates present in the training grid.

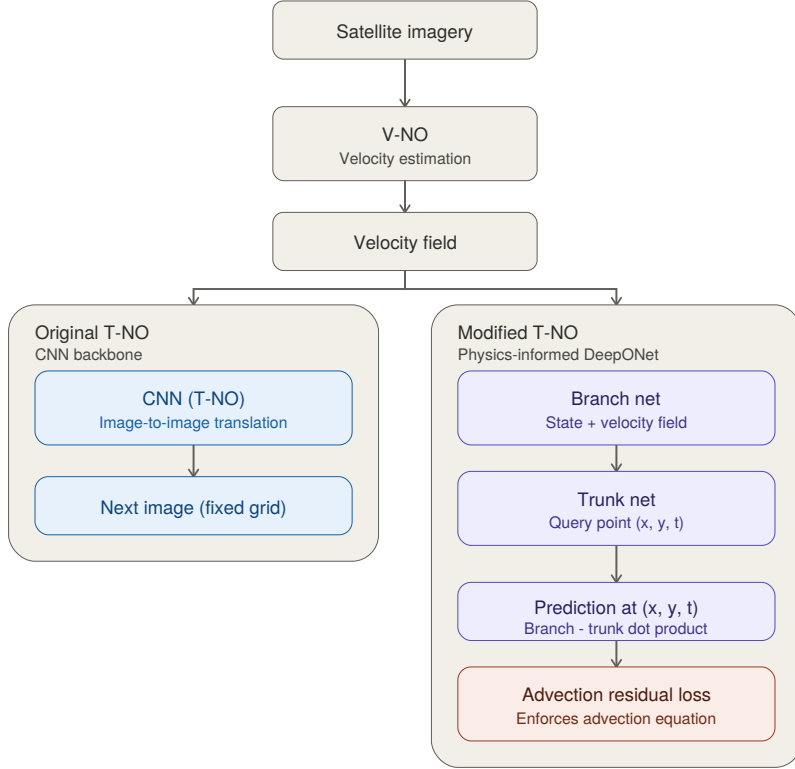


Figure 1: Modified PIANO architecture. The shared V-NO stage (top) is unchanged. The original CNN-based T-NO (left) maps directly to a fixed-grid next image. The proposed T-NO (right) is a physics-informed DeepONet, in which a branch net encodes the current state and velocity field, a trunk net encodes an arbitrary space-time query coordinate (x, y, t) , and their combination is regularized by an advection-residual physics loss.

3 Stage 1 — V-NO (Unchanged)

The V-NO remains a convolutional encoder that maps a short sequence of past satellite frames to a dense velocity field:

$$\mathbf{v}(x, y) = \text{V-NO}(I(\cdot, \cdot, t_{-k}), \dots, I(\cdot, \cdot, t_0)) \quad (2)$$

Implementation, loss, and training schedule for V-NO are retained from the original PIANO paper (no change proposed here).

4 Stage 2 — T-NO as a Physics-Informed DeepONet

4.1 Architecture

The modified T-NO follows the standard DeepONet decomposition into a **branch net** and a **trunk net**, whose outputs are combined via an inner product.

Branch net $\mathcal{B}_\theta$

- Input: the current state $I(\cdot, \cdot, t_0)$ and velocity field $\mathbf{v}(\cdot, \cdot)$, both sampled at a fixed set of m sensor locations on Ω .
- A CNN encoder (e.g. a small ResNet stem) first compresses the raw image and velocity channels into a feature map, which is flattened and passed through fully connected layers.
- Output: a latent coefficient vector $\mathbf{b} \in \mathbb{R}^p$.

$$\mathbf{b} = \mathcal{B}_\theta(I(\cdot, \cdot, t_0), \mathbf{v}(\cdot, \cdot)) \quad (3)$$

Trunk net $\mathcal{T}_\phi$

- Input: a query coordinate (x, y, t) — any spatial location and any future lead time, not restricted to the training grid.
- A small MLP (with sinusoidal/Fourier positional encoding of (x, y, t) recommended, to help the network represent high-frequency spatial detail).
- Output: a latent basis vector $\boldsymbol{\tau} \in \mathbb{R}^p$.

$$\boldsymbol{\tau} = \mathcal{T}_\phi(x, y, t) \quad (4)$$

Combination (prediction)

$$\hat{s}(x, y, t) = \sum_{k=1}^p b_k \tau_k + b_0 \quad (5)$$

where b_0 is a bias term (standard DeepONet practice).

4.2 Why This Preserves PIANO’s Conceptual Split

- The branch net still plays the role of “what is the current situation” (state + velocity), matching T-NO’s original input.
- The trunk net replaces the fixed output grid with a continuous query mechanism — the model is now a function of (x, y, t) rather than a fixed-size decoder.

5 Physics-Informed Loss

5.1 Data (Reconstruction) Loss

Standard supervised term against ground-truth satellite/precipitation fields at the sampled training query points:

$$\mathcal{L}_{\text{data}} = \frac{1}{N} \sum_{i=1}^N \left(\hat{s}(x_i, y_i, t_i) - s^{\text{true}}(x_i, y_i, t_i) \right)^2 \quad (6)$$

5.2 Physics (Advection Residual) Loss

The governing assumption is that the field is transported by the estimated velocity field, i.e. it approximately satisfies the advection equation:

$$\frac{\partial s}{\partial t} + u(x, y) \frac{\partial s}{\partial x} + w(x, y) \frac{\partial s}{\partial y} = 0 \quad (7)$$

Because $\hat{s}(x, y, t)$ is produced by a DeepONet with a differentiable trunk net, the required partial derivatives $\partial \hat{s} / \partial t$, $\partial \hat{s} / \partial x$, $\partial \hat{s} / \partial y$ can be obtained by automatic differentiation with respect to the trunk net’s input coordinates — exactly as in the physics-informed DeepONet formulation used for PDE operator learning.

The residual is evaluated at a set of N_r collocation points (x_j, y_j, t_j) sampled densely (and randomly) in space-time, not just at labeled data points:

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_r} \sum_{j=1}^{N_r} \left(\left. \frac{\partial \hat{s}}{\partial t} \right|_j + u_j \left. \frac{\partial \hat{s}}{\partial x} \right|_j + w_j \left. \frac{\partial \hat{s}}{\partial y} \right|_j \right)^2 \quad (8)$$

5.3 Combined Objective

$$\mathcal{L}_{\text{total}} = (1 - \alpha) \mathcal{L}_{\text{data}} + \alpha \mathcal{L}_{\text{phys}} \quad (9)$$

$\alpha \in [0, 1]$ is the physics-weighting hyperparameter, consistent with PIANO’s original sensitivity-analysis variable, and should be tuned the same way (sweep and select based on validation CSI/MSE).

Optional, following the physics-informed DeepONet paper’s practice: add a boundary/initial-condition loss term $\mathcal{L}_{\text{ic}}$ enforcing $\hat{s}(x, y, t_0) = I(x, y, t_0)$, to anchor the operator to the known initial state.

$$\mathcal{L}_{\text{total}} = (1 - \alpha) \mathcal{L}_{\text{data}} + \alpha \mathcal{L}_{\text{phys}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}} \quad (10)$$

6 Training Procedure

1. **Pretrain / freeze V-NO** (or train jointly — follow original PIANO’s chosen strategy) to obtain $\mathbf{v}(x, y)$ for each training sample.
2. **Sample sensor points** for the branch net input (fixed sensor grid, e.g. a downsampled version of Ω).
3. **Sample query points** for the trunk net:
 - Labeled points (x_i, y_i, t_i) at available ground-truth lead times (1h–8h, matching PIANO’s evaluation horizons) for $\mathcal{L}_{\text{data}}$.
 - Random collocation points (x_j, y_j, t_j) spanning continuous space-time (including non-integer lead times) for $\mathcal{L}_{\text{phys}}$.
4. **Forward pass:** compute $\mathbf{b}$ once per sample from the branch net; evaluate the trunk net at all sampled query points; combine to get predictions.
5. **Backward pass:** compute $\mathcal{L}_{\text{data}}$, $\mathcal{L}_{\text{phys}}$ (via autodiff through the trunk net w.r.t. x, y, t), and optionally $\mathcal{L}_{\text{ic}}$; backpropagate the weighted sum $\mathcal{L}_{\text{total}}$.
6. **Optimizer:** Adam with learning-rate warmup/decay, consistent with standard DeepONet training recipes; gradient clipping recommended since second-order-adjacent derivative terms can be noisy early in training.
7. **α scheduling** (optional improvement over a fixed α): start with a low α (data-dominated) and anneal upward, so the network first learns a reasonable fit before being constrained by the physics residual — mirrors curriculum strategies used in PINN training.

7 Inference

At inference time, given a new satellite sequence:

1. Run V-NO to obtain $\mathbf{v}(x, y)$.
2. Run the branch net once to obtain $\mathbf{b}$.
3. Query the trunk net at **any** desired (x, y, t) — arbitrary lead time (not restricted to 1h/2h/.../8h bins), arbitrary spatial resolution, or a cropped region of interest — without retraining or re-running the branch net.

This is the key practical advantage over the CNN T-NO: one branch-net forward pass amortizes across unlimited query resolutions/horizons.

8 Evaluation

Retain PIANO’s original evaluation protocol for direct comparability:

- **Critical Success Index (CSI)** at each lead time (1h–8h), computed by querying the trunk net at the corresponding integer-hour coordinates and rasterizing to the original grid for scoring against ground truth.

- **MSE** across lead times.
- **α sensitivity analysis**, replicated for the new formulation, to check whether the optimal physics weight shifts once the operator is coordinate-based rather than grid-based.
- **New metric enabled by this architecture**: prediction consistency across non-integer lead times (e.g. 2.5h) and across resolutions, to quantify the resolution-independence benefit — something the CNN baseline cannot produce natively.
- **Baselines**: compare against the original CNN-based PIANO, plus NPM and PhyDNet, as in the original paper.

9 Summary of Changes vs. Original PIANO

Component	Original PIANO	Modified PIANO
V-NO	CNN velocity encoder	Unchanged
T-NO	CNN image-to-image (ResNet/pix2pix-style)	Physics-informed DeepONet (branch + trunk)
Output	Fixed-grid image per lead-time bin	Continuous function of (x, y, t)
Physics constraint	α -weighted advection loss on grid outputs	α -weighted advection residual via autodiff on trunk-net coordinates
Query flexibility	Fixed resolution, fixed lead-time bins	Arbitrary resolution, arbitrary lead time
Added cost	—	Trunk-net autodiff overhead; collocation-point sampling

Table 1: Summary of architectural and training differences between the original and modified PIANO frameworks.